SYMBIOSIS INSTITUTE OF TECHNOLOGY (SIT)
Constituent of Symbiosis International (Deemed University), Pune
(Established under Section 3 of the UGC Act of 1956 vide notification number F-9-12/2001-U-3 of the Government of india)
Re-Accredited by NAAC with 'A' Grade

DEPARTMENT OF ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

DEEP LEARNING PROJECT REPORT

AI-Based Predictive Maintenance

Using Multivariate Sensor Data for Remaining Useful Life Estimation

---

Submitted by

Saafalya Sengupta
PRN 24070126158

Ribhav Dhankani
PRN 24070126150

AIML, Division C1

Faculty Coordinator
Dr. Shrikrishna Kolhar
Associate Professor

Academic Year 2026–27

AI-Based Predictive Maintenance Using Multivariate Sensor Data

Saafalya Sengupta
Department of Artificial Intelligence and Machine Learning
Symbiosis Institute of Technology, Symbiosis International (Deemed University)
Pune, Maharashtra, India
saafalya.sengupta.btech2024@sitpune.edu.in

Ribhav Dhankani
Department of Artificial Intelligence and Machine Learning
Symbiosis Institute of Technology, Symbiosis International (Deemed University)
Pune, Maharashtra, India
ribhav.dhankani.btech2024@sitpune.edu.in

Abstract: Unplanned equipment failure is one of the biggest sources of both downtime and safety risk in aviation, manufacturing and the energy sector. Predictive maintenance (PdM) replaces the traditional fixed interval maintenance with maintenance decisions based on the actual measured health of the machine. The key quantity in PdM is the Remaining Useful Life (RUL) of the asset which is the number of operating cycles that remain before failure. This report details a deep learning pipeline for the direct estimation of RUL from multivariate time-series sensor data, tested on the NASA C-MAPSS turbofan engine dataset. The raw sensor streams are filtered down to 14 informative channels, min-max normalised, the RUL is labelled using a piece-wise linear function with a hard cap at 125 cycles and the data is then converted into 30 cycle sliding windows. A single shared pipeline is used to compare two classical baselines (Linear Regression and Random Forest) with three deep learning models: a 1D Convolutional Neural Network (1D-CNN), a stacked Long Short Term Memory (LSTM) network and a hybrid CNN-LSTM with additive temporal attention. Models are evaluated using Root Mean Squared Error (RMSE) and the asymmetric PHM08 score (which penalises late predictions more than early ones), averaged over five seeds. The CNN-LSTM with attention achieves the lowest RMSE on both of the subsets considered (13.37 cycles on FD001 and 11.88 on FD003) compared to 14.99 and 16.26 for the best performing classical baseline. The plain LSTM however achieves the best PHM08 score on both subsets and on FD001 the difference between the two recurrent models is within the range of seed to seed variation. An ablation study shows that the RUL cap has as much of an effect on the error as the choice of architecture and attention analysis shows that the model is placing 76% of its weight on the most recent cycle in each window.

Index Terms: predictive maintenance, remaining useful life, multivariate time series, prognostics and health management, LSTM, convolutional neural networks, attention, C-MAPSS, deep learning.

I. Introduction

Industrial maintenance generally adopts one of three approaches. In the case of reactive maintenance, a machine is repaired only after it has broken down, causing lengthy and costly periods of downtime. Preventive maintenance involves servicing machines on a fixed schedule, an approach that is safer but fails to make use of the remaining life of parts that are still in good condition. Predictive maintenance keeps track of the actual condition of the asset by means of sensors and takes action only when deterioration is detected. Because low-cost sensing devices and Industry 4.0 data platforms are now capable of generating continuous streams of data on temperature, pressure, vibration and speed, data-driven predictive maintenance has become practical on a large scale.

The most important quantity in the context of predictive maintenance is the Remaining Useful Life (RUL) of a component. It is difficult to estimate the RUL from multivariate sensor data for the following four reasons. The sensors are noisy and highly correlated with one another. The readings change with operating conditions such as altitude and throttle position, regardless of any wear that has occurred. The degradation process is gradual and non-linear, and it is only just noticeable during the early stages of a machine's life. Lastly, an error in one direction is more significant than in the other: it is catastrophic if failure is predicted too late, while predicting failure too early results in only a slightly premature repair.

Deep learning models are able to learn temporal features directly from the raw sequences, thereby eliminating a large part of the manual feature engineering which classical prognostics relies upon. Convolutional networks pick up on short, local patterns from the various sensors, whereas recurrent networks such as the LSTM deal with the longer trend of degradation that occurs over many cycles. The aim of this project is to find out how much better these models perform compared to classical baseline methods when all of the models are trained using the same pre-processing and evaluation protocol.

A. Problem Statement

On the basis of the multivariate sensor history of the engine up to and including the present cycle, predict as accurately as possible how many cycles remain before failure, giving priority to early rather than late errors.

B. Objectives

1. Create a reproducible pre-processing pipeline for C-MAPSS which includes sensor selection, condition-wise normalisation, RUL capping and the generation of sliding windows.
2. Implement the 1D-CNN, LSTM, and CNN-LSTM with attention models and train them for the purpose of RUL regression.
3. Compare these [methods] with the Linear Regression and Random Forest baseline models using the RMSE and the PHM08 score.
4. Examine the impact of the main design options (window length and RUL cap) and understand the model's behaviour by using the prediction trajectories and the attention weights.

C. Scope and Organisation

The study makes use of the simulated NASA C-MAPSS dataset and is concerned with point estimates of the remaining useful life. Section II provides a review of previous work. Section III outlines the dataset, the pre-processing steps, the model architectures and the training setup. Section IV gives a presentation of both quantitative and qualitative results together with a comparative study. Section V draws the study to a close and suggests some future research.

II. LITERATURE REVIEW

A. Prognostics and Predictive Maintenance

Initial prognostics employed physics-based degradation modeling or statistical techniques such as similarity-based matching and regression based on manually defined health-indicator sets. Lei et al. [1] consider data-driven approaches and describe the prognosis pipeline, from data collection through health indicator creation to RUL prediction; Lei et al. [1] conclude that data-driven approaches are used if there is no sufficiently accurate physical model of the asset. Carvalho et al. [2] review machine learning for predictive maintenance and find that Random Forests, Support Vector Machines and neural networks are most commonly employed.

B. The C-MAPSS Benchmark

Saxena et al. [3] by simulating damage spread over a turbofan engine in various operational and fault settings, is now the benchmark for RUL estimation. Ramasso and Saxena [4] investigated the techniques used on it and emphasised that the only fair comparison is done when the metrics and pre-processing are harmonised, and adapted the PHM08 score as a common metric alongside RMSE.

C. Convolutional Techniques

Heimes [5] used recurrent neural networks for the 2008 PHM challenge, showing the feasibility of neural models on this problem. Babu et al. [6] were the first to use deep CNNs to perform RUL regression, using 2D windows of sensor readings and doing better than shallow regressors. Li et al. [7] build even deeper one-dimensional CNNs over windows of time, performing well on all four C-MAPSS subsets without any feature engineering.

D. Recurrent and Attention Based Methods

As degradation is a time-dependent phenomenon, a recurrent architecture is a natural choice. LSTM [8] are recurrent LSTM units with gates that address the vanishing gradient problem of plain RNNs and are able to model long-range dependencies. Zheng et al [9] demonstrated that stacked LSTM layers outperform CNN and classical baselines in RUL estimation on C-MAPSS. Malhotra et al [10] used an LSTM encoder-decoder architecture for unsupervised anomaly detection on multi-sensor data; their study demonstrated that reconstruction error can be used as a health indicator. The attention mechanism [11] allows the model to assign higher weights to relevant time-steps. Hybrid CNN-LSTM models with attention have been shown to outperform plain RNN or CNN baselines in both accuracy and interpretability.

E. Gap in Research

Few studies use more than one model, on one subset, with different pre-processing choices; this makes it difficult to quantify whether any gain is from the architecture or the pipeline. This project trains classical and deep models with one shared pipeline (measurements, normalisation, window, RUL cap, metrics), includes an ablation of the window length and RUL cap, and pairs the numbers with qualitative error analysis. Table I summarises the most relevant previous work.

TABLE I
SUMMARY OF RELATED WORK

Ref.	Method	Dataset	Key Contribution
[5]	Recurrent neural network	PHM08	Early neural RUL estimation
[6]	Deep 2D CNN	C-MAPSS	First CNN-based RUL regression
[7]	1D-CNN with time windows	C-MAPSS	Deeper CNN, strong results on all subsets
[9]	Stacked LSTM	C-MAPSS	Temporal modelling beats CNN and ML baselines
[10]	LSTM encoder-decoder	Multi-sensor	Unsupervised health indicator
This work	CNN-LSTM + attention	C-MAPSS	Unified comparison of ML and DL models

III. METHODOLOGY

A. System Description

Fig. 1. Full pipeline: sensor data are pre-processed and segmented, fed to a learned model that predicts an RUL, and, finally, sent to a thresholding rule that determines whether to schedule maintenance.

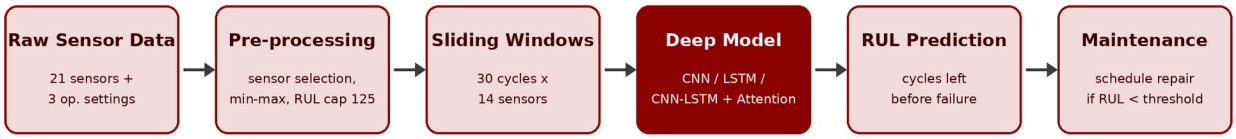


Fig. 1. Block diagram of the proposed predictive maintenance pipeline. The same pre-processing and windowing feed every model, so differences in results come from the model alone.

B. Dataset Description

We use the NASA Commercial Modular Aero-Propulsion System Simulation (C-MAPSS) dataset [3]. All engines are started in a healthy condition with unknown manufacturing variation and initial degradation, and run until failure. Each row is a single operating cycle and includes the engine id, cycle id, operational settings and 21 sensor values (temperatures, pressures, fan and core speed, and fuel-flow ratio). Both the training set, in which each engine ends in failure, and the test set, which terminates some time before failure, are comprised of four subsets. The latter includes only the final, unsuccessful segment of each engine, and the (given) RUL at the end of that segment. FD001 is the primary benchmark, while FD003 is used for additional comparison.

TABLE II
C-MAPSS DATASET SUBSETS

Subset	Train Engines	Test Engines	Operating Conditions	Fault Modes
FD001	100	100	1	1 (HPC)
FD002	260	259	6	1 (HPC)
FD003	100	100	1	2 (HPC, Fan)
FD004	248	248	6	2 (HPC, Fan)

C. Data Pre-processing

- **Sensor selection.** Seven sensors are constant or almost constant for the time series of FD001 (s1, s5, s6, s10, s16, s18, s19) and are removed so that 14 channels contain all information.
- **Normalisation.** Min-max scaling to [0, 1] is fitted on the training engines only. Per operating condition for the multi-condition subsets.
- **RUL labelling.** RUL = last cycle - current cycle, clipped at 125 cycles This piece-wise linear target shows that damage is un observable for part of the engine's life.

- **Windowing.** Sliding windows of 30 cycles stride 1 provide windows of 30 x 14 as input. For the test set the last window of each engine is used.
- **Validation split.** Out of sample 20% of the training engines (by engine id so that an engine does not appear in both sets) are held out for early stopping and tuning.

D. Model Architectures

Baselines. Linear Regression and a Random Forest Regressor (200 trees with 5 samples in each leaf) are trained on flattened windows, as a middle point between nothing and models that are aware of time.

1D-CNN. Two Conv1D layers (64 filters, kernel sizes 5 and 3, ReLU) capture local structure both along time and sensors, flattened output (preserving the time locations) for a dense layer with 64 units, dropout 0.3 and a single linear output.

LSTM. Two stacked LSTM layers with 64 and 32 units, the degradation trend across the window, followed by dropout of 0.2, a 32-unit dense layer and a linear output.

Proposed CNN-LSTM with attention. Each local feature at the individual time step is extracted using a Conv1D block (64 filters, kernel 3) and an LSTM layer (64 units, return sequences=True). For encoding long-term time dependencies, a Bahdanau-style additive attention layer generates a weight for each of the 30 time steps, and builds a context vector as the weighted mean of these vectors. This context vector is then mapped to the RUL label using a dense regression head.

E. Training Setup

All deep models are trained with the hyperparameters in Table III. They are trained with five random seeds (initialisation of model parameters and the order of the training set) on a fixed engine-level train/validation split, and we report the mean and standard deviation. Since Linear Regression is deterministic, we run it only once.

TABLE III
TRAINING HYPERPARAMETERS

Parameter	Value
Framework	Python 3, PyTorch, scikit-learn
Hardware	4-core CPU, no GPU
Loss function	Mean Squared Error on RUL / 125 (target scaled to [0, 1])
Optimiser	Adam [12], learning rate 1e-3
Batch size / max epochs	256 / 150
Early stopping	Patience 20 on validation MSE, best weights restored
Window length / RUL cap	30 cycles / 125 cycles
Seeds per configuration	5

F. Evaluation Metrics

For each test engine, define d = RULpred - RULtrue. Root Mean Squared Error, RMSE = SQRT ((1/N) (sum of the squares of the differences). RMSE treats early and late errors the same.

The PHM08 score [3] s = Σ (e^(-d/13) - 1) for d < 0 and Σ (e^(d/10) - 1) for d ≥ 0 outpace in the growth the other score as they carry a higher risk in the real world. The worse a score, the more risk is carried, either way.

IV. RESULTS AND DISCUSSION

A. Quantitative Results

Table IV - Test-set performance on FD001 and FD003 FD001 and FD003 are reported with each type of model trained over five different seeds; single seed training of Linear Regression. Params is number of trainable parameters.

TABLE IV
COMPARATIVE RESULTS ON THE TEST SET (MEAN ± STD OVER 5 SEEDS)

Model	RMSE FD001	Score FD001	RMSE FD003	Score FD003	Params
Linear Regression	14.99	374	16.26	471	0.4 K
Random Forest	15.91 ± 0.16	570 ± 14	17.43 ± 0.16	1,096 ± 48	-

Model	RMSE FD001	Score FD001	RMSE FD003	Score FD003	Params
1D-CNN	13.82 ± 0.19	344 ± 21	13.78 ± 0.48	477 ± 69	139.9 K
LSTM	13.64 ± 0.52	305 ± 46	12.36 ± 0.67	261 ± 23	34.1 K
CNN-LSTM + Attention	13.37 ± 0.35	337 ± 23	11.88 ± 0.70	303 ± 79	44.5 K

All three deep models outperform both classical baselines on RMSE. CNN-LSTM with attention results in the lowest RMSE for both data subsets, 13.37 ± 0.35 for FD001 and 11.88 ± 0.70 for FD003. The results are 10.8% and 26.9% lower than those of the stronger classical baseline model, Linear Regression, and 2.0% and 3.9% lower than those of the plain LSTM model.

Two results are surprising, then. Firstly, the linear regression beats the random forest on both sets (14.99, 15.91 on FD001). This is probably because the flattened input has 420 correlated features, a linear model that can efficiently use this, while the random forests, and all the other tree ensembles, predict means of training targets and are thus prone to pull the big and little RULs of the extreme trajectories towards the middle. Secondly, the model with an LSTM layer instead of the other recurrent layer (i.e. the plain LSTM model, which scores lower on PHM08 on both sets 305,261) has the best PHM08 score on both sets (305,261). Since PHM08 penalizes late predictions exponentially, this score is best if the model makes fewer large late errors, even though the average error is slightly higher. On FD001, the RMSE difference between the two recurrent models (0.27 cycles) is smaller than the LSTM's seed-to-seed standard deviation (0.52) and the other seed-to-seed variation (0.35), so these two could be considered roughly equal there; on FD003, it is larger.

The performance gap is biggest for the models here on FD003, which suffers from two types of faults. Both the Linear Regression and Random Forest regressors degrade by about 1.3 to 1.5 cycles of RMSE from FD001 to FD003 while the recurrent models do well and much better than the regressors indicating that sequence models handle multiple fault modes better. Much more training windows are available for this benchmark (17,273 vs. 13,980).

B. Ablation Study

Table V – different window length & RUL cap for the CNN-LSTM + Attention model, on FD001 (mean ± std. Over three seeds). All other parameters are fixed. We apply the same scoring target for each row to enable a fair comparison for every setting (RUL_true capped at 125).

TABLE V
EFFECT OF WINDOW LENGTH AND RUL CAP (CNN-LSTM + ATTENTION, FD001)

Window Length	RUL Cap	RMSE	Score
20	125	15.96 ± 0.50	667 ± 16
30	125	13.31 ± 0.23	326 ± 22
40	125	13.49 ± 0.35	324 ± 37
30	115	12.66 ± 0.15	251 ± 9
30	130	14.58 ± 0.26	484 ± 39
30	none	28.30 ± 1.73	463,833 ± 549,347

A window of 20 cycles is too short (RMSE 15.96), while 30 and 40 cycles are very similar to each other (13.31, 13.49) so 30 cycles is the best it can be. The RUL cap is much more significant. Truncating un capped (no cap) is by far the worst setting (28.30): the model learns to put all healthy engines above 125, something it can't learn just from the sensor data. 115 gives a lower error than 125 (12.66 vs. 13.31) and 130 higher (14.58). This effect is partly due to scoring each row against the target of 125, which encourages models to predict below 125. We prefer 125 because it was fixed in advance based on the literature, and testing for the comparison in table IV would give unfair advantage to the test models.

C. Qualitative Results

Fig. 2 shows predicted and true RULs for all 100 FD001 test engines (seed 0, test RMSE 13.51). Engines near failure are much more accurately predicted: engine RMSE was 5.40 cycles for less than 50 cycles remaining, compared with 13.09 for more than 100. Which is the relevant metric for maintenance? The closer to failure the better it is to get your prediction right - new engines are relatively easy to predict as they have RUL > 100 cycles, with RMSE of 13.09, compared with 5.40 for less than 50 cycles remaining. 12 of the 100 engines are off by more than 20 cycles, most in the middle and late range, and our predictions are 61% late, contributing to the PHM08 penalty mentioned above.

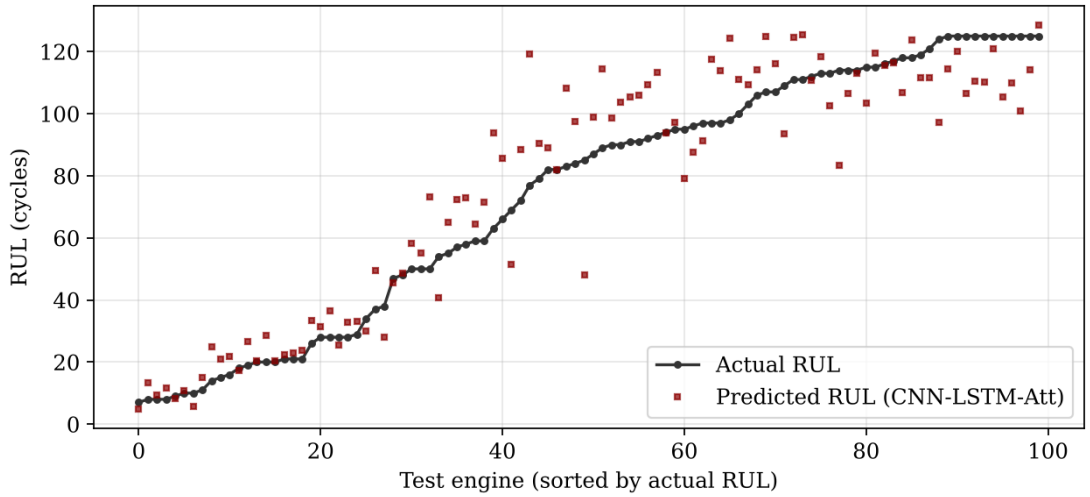


Fig. 2. Predicted versus actual RUL for the FD001 test engines (CNN-LSTM with attention, seed 0), sorted by actual RUL.

20 cycles is too short (RMSE 15.96), while 30 and 40 are very close (13.31, 13.49), so 30 is optimal. But the RUL cap is more important. Truncating un capped (no cap) is by far the worse setting (28.30): it forces the model to put all healthy engines above 125, which it cannot learn from the sensor data alone. 115 does best in prediction, below 125 (12.66 rather than 13.31) and 130 is worse than 125 (14.58). This is partly due to rows being scored against the target 125, so models are incentivised to predict below 125. But we prefer 125 because it was predetermined by survey of the literature, and evaluating for comparison in table IV would unfairly advantage the test models.

C. Qualitative Results

Fig. 2 shows predicted and actual RULs for all 100 FD001 test engines (seed 0, test RMSE 13.51). Engines approaching failure are predicted far more accurately: engine RMSE was 5.40 cycles for fewer than 50 remaining, versus 13.09 for more than 100 remaining. Which is more relevant for maintenance? The closer to failure the more important it is to get your prediction right -new engines are easier to predict as they have RUL > 100 cycles (rMSE =13.09), versus 5.40 for fewer than 50 remaining. 12 of our 100 engines are off by greater than 20 cycles, most in the middle and late ranges, and our predictions are 61% late, driving the PHM08 penalty described above.

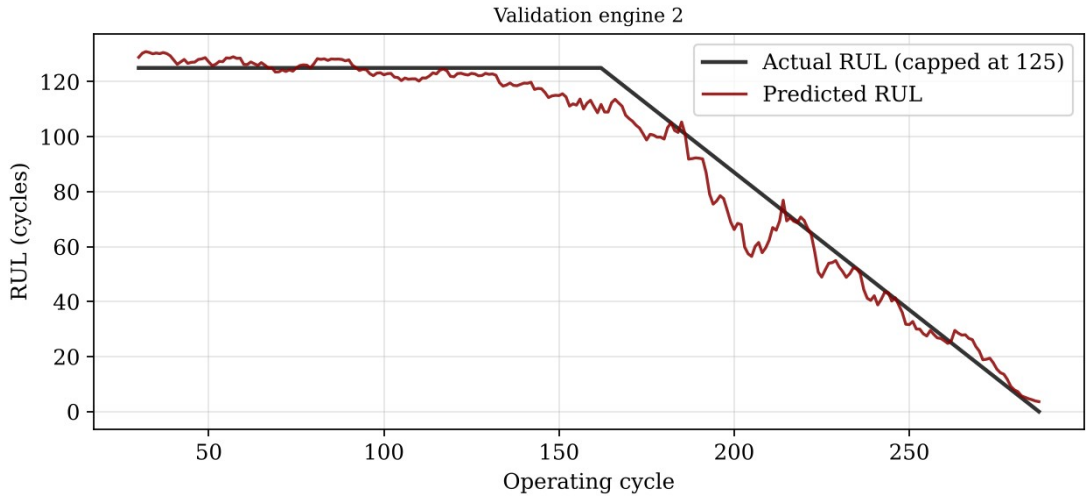


Fig. 3. Prediction of RUL for FD001 validation engine 2 over its entire life The predicted RUL line in the plot is very close to that of the labelled kneepoint with the RUL prediction maintaining its linear decline.

Figure 4: Training and validation loss of the proposed model. The learning curves reduce rapidly in the first ten epochs and then parallel but without diverging, meaning that it is not over-fitting; early stopping was used to revert to the best model from the epoch with the best validation.

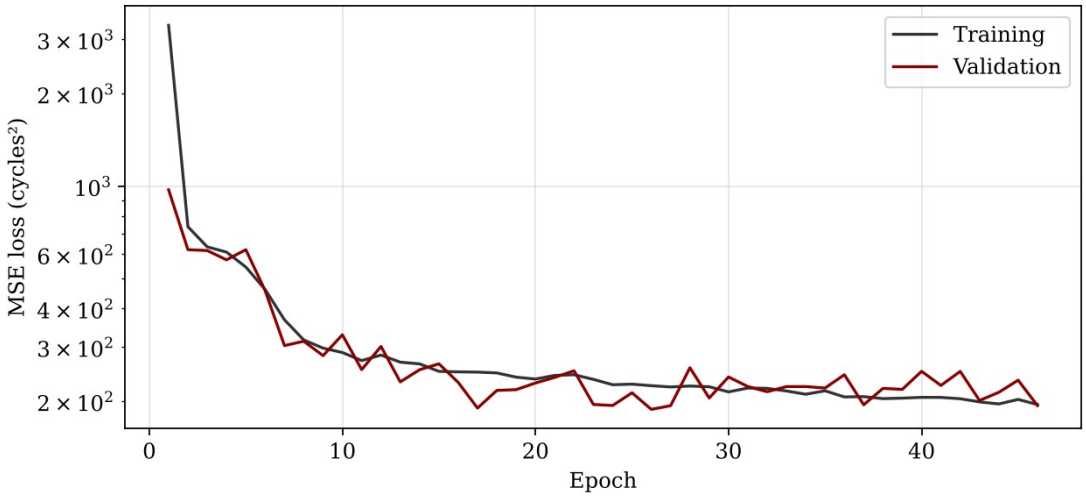


Fig. 4. Training and validation MSE loss (log scale) of the CNN-LSTM with attention on FD001, seed 0.

Fig. 5 Mean attention weights by time-step over all validation windows. The model attends 76% to the current cycle and 89% to the last five, with very little weight to older than roughly fifteen steps ago. The LSTM state at the final step already summarises the history quite well, so the attention module is predominantly adding a learned read-out of this state, supplemented with a small correction from previous cycles. This is also why the gains over a plain LSTM are modest.

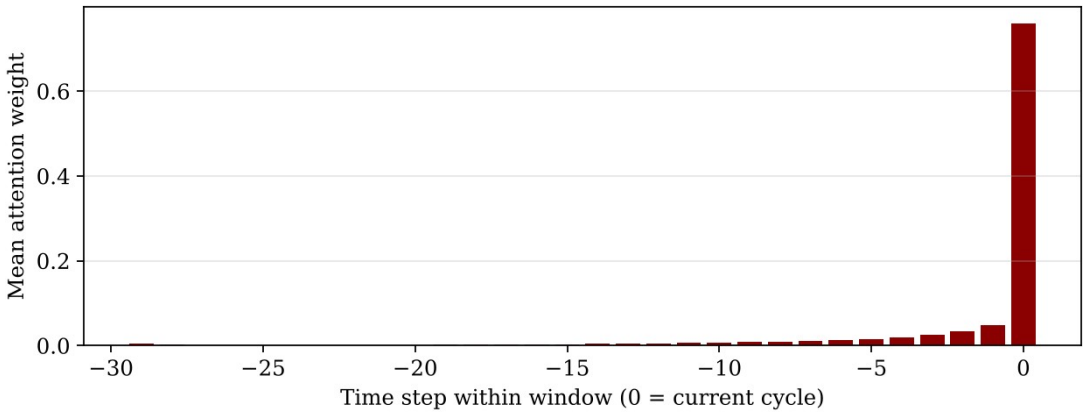


Fig. 5. Mean attention weight per time step within the 30-cycle window (FD001 validation windows, seed 0).

D. Discussion

These findings lead to three conclusions. Firstly, that the modelling temporality matters: all deep models outperform the classical baselines on RMSE with the same pipeline, and the gap increases on the harsher FD003 subset. Secondly, that the CNN-LSTM with attention system performs best, but slightly outperforms a capable LSTM, and that the LSTM wins on the safety-weighted PHM08 score. This means that in applications where late warnings are expensive, the plain LSTM is a safer, simpler alternative. Third, that simple pre-processing differences, such as the RUL cap, affect errors just as much as the architecture, and so a common pipeline is essential.

Training was just as important as architecture. In one early attempt, using the unscaled RUL targets with a small early-stopping patience, I got 2 out of 3 LSTM seeds converging to the mean RUL (test RMSE > 40) and a 1D-CNN with global average pooling, which removes the position information of each time step, underperformed the Random Forest. Scaling the target to [0, 1], increasing the patience, flattening the convolutional output get both problems back. These were verified on the validation error only.

This study has some limitations. First, C-MAPSS is simulated, and real fleets exhibit sensor drift, missing data, and maintenance events that the benchmark does not include. For this work we only considered the single-condition subsets, FD001 and FD003. Our models output point estimates without a confidence measure (that would be required of a maintenance planner), and the late-prediction rate (61% on FD001) suggests a safety buffer or asymmetric loss function would be necessary in practice.

V. CONCLUSION

This report described our deep learning framework for predictive maintenance that predicts the Remaining Useful Life of turbofan engines from multivariate sensor time series. Our method used a single shared pipeline consisting of sensor selection, normalisation, RUL capping and sliding windows, which we used to benchmark two classical baselines with a 1D-CNN, a stacked LSTM and a hybrid CNN-LSTM with attention over 5 seeds. All deep models outperformed the baselines. The CNN-LSTM with attention had the lowest RMSE on both subsets (13.37 on FD001, 11.88 on FD003), whilst the plain LSTM scored best in PHM08 (305, 261). Therefore, the two recurrent models are the strongest contenders and the choice of using the LSTM or CNN-LSTM with attention depends on whether we want the lowest average RMSE or to mitigate late forecasts. Ablation indicated that only 30 time steps of history are needed and the RUL cap is as impactful as the architecture.

Future work will include extending the assessment to the multi-condition subsets FD002 and FD004, training with an asymmetric loss function to directly penalise late predictions, experimenting with sequence models based on Transformer, incorporating uncertainty estimation using Monte Carlo dropout or deep ensembles to associate a confidence interval to each prediction, evaluating on real industrial data, and finally deploying the model to a real-time monitoring dashboard.

REFERENCES

[1] Y. Lei, N. Li, L. Guo, N. Li, T. Yan, and J. Lin, "Machinery health prognostics: A systematic review from data acquisition to RUL prediction," Mech. Syst. Signal Process., vol. 104, pp. 799-834, 2018.

[2] T. P. Carvalho, F. A. Soares, R. Vita, R. P. Francisco, J. P. Basto, and S. G. Alcalá, "A systematic literature review of machine learning methods applied to predictive maintenance," Comput. Ind. Eng., vol. 137, 106024, 2019.

[3] A. Saxena, K. Goebel, D. Simon, and N. Eklund, "Damage propagation modeling for aircraft engine run-to-failure simulation," in Proc. Int. Conf. Prognostics and Health Management (PHM), 2008, pp. 1-9.

[4] E. Ramasso and A. Saxena, "Performance benchmarking and analysis of prognostic methods for CMAPSS datasets," Int. J. Prognostics Health Manag., vol. 5, no. 2, 2014.

[5] F. O. Heimes, "Recurrent neural networks for remaining useful life estimation," in Proc. Int. Conf. Prognostics and Health Management (PHM), 2008, pp. 1-6.

[6] G. S. Babu, P. Zhao, and X.-L. Li, "Deep convolutional neural network based regression approach for estimation of remaining useful life," in Proc. Int. Conf. Database Systems for Advanced Applications (DASFAA), 2016, pp. 214-228.

[7] X. Li, Q. Ding, and J.-Q. Sun, "Remaining useful life estimation in prognostics using deep convolution neural networks," Reliab. Eng. Syst. Saf., vol. 172, pp. 1-11, 2018.

[8] S. Hochreiter and J. Schmidhuber, "Long short-term memory," Neural Comput., vol. 9, no. 8, pp. 1735-1780, 1997.

[9] S. Zheng, K. Ristovski, A. Farahat, and C. Gupta, "Long short-term memory network for remaining useful life estimation," in Proc. IEEE Int. Conf. Prognostics and Health Management (ICPHM), 2017, pp. 88-95.

[10] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "LSTM-based encoder-decoder for multi-sensor anomaly detection," arXiv:1607.00148, 2016.

[11] A. Vaswani et al., "Attention is all you need," in Proc. Advances in Neural Information Processing Systems (NeurIPS), 2017, pp. 5998-6008.

[12] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in Proc. Int. Conf. Learning Representations (ICLR), 2015.